



MCAST

Hybrid Vision-Based Traffic Anomaly Detection Using Vehicle Tracking and Speed Estimation

Denzel Baldacchino

Supervisor: Marco Farrugia

June - 2026

A dissertation submitted to the Institute of Information and Communication Technology in partial fulfilment of the requirements for the degree of BSc (Hons) Multimedia in Software Development

Authorship Statement

This dissertation is based on the results of research carried out by myself, is my own composition, and has not been previously presented for any other certified or uncertified qualification.

The research was carried out under the supervision of Marco Farrugia

.....

Date

.....

Signature

Copyright Statement

In submitting this dissertation to the MCAST Institute of Information and Communication Technology, I understand that I am giving permission for it to be made available for use in accordance with the regulations of MCAST and the Library and Learning Resource Centre. I accept that my dissertation may be made publicly available at MCAST's discretion.

.....

Date

.....

Signature

Acknowledgements

The list of people that the Student would like to thank on the completion of the dissertation. Mr Marco Farrugia, who supported me during my dissertation work as my tutor.

Abstract

This section should clearly state what the study is about, summarizing how it was carried out and what the results were. References are not to be included in the abstract. It should present only the essentials of the work in general.

Keywords: Dissertation, keywords.

Table of Contents

Authorship Statement	i
Copyright Statement	ii
Acknowledgements	iii
Abstract	iv
List of Figures	vii
List of Tables	viii
List of Abbreviations	ix
1 Introduction	1
1.1 Sub-chapter One	1
1.2 Sub-chapter Two	3
1.3 Sub-chapter Three	4
1.4 Sub-chapter Four	5
1.5 Sub-chapter Five	6
2 Literature Review	8
2.1 Overview	8
2.2 One	8
2.3 Two	9
2.4 Three	10
2.4.1 Four	11
2.4.2 Five	11
3 Research Methodology	13
3.1 Introduction to the Methodology	13
3.2 Data Engineering, Harmonisation, and Mathematical Balancing . . .	14
3.3 Data Acquisition and Mathematical Camera Stabilisation	16
3.4 Spatial Perception: YOLO11 and Implicit BoT-SORT Tracking . . .	17
3.5 GDPR Compliance via Gaussian Obfuscation	18
3.6 3D Metric Reconstruction and Speed Estimation	19
3.7 Kinematic Trajectory Smoothing via Kalman Filtering	21
3.8 Spatiotemporal Kinetic Energy Mapping via Dense Optical Flow .	22
3.9 Deep Learning Anomaly Classification	23
3.10 Asynchronous Dual-Threaded Architecture and System Integration .	25
3.11 Evaluation of Results	26

4	Analysis, Results and Discussion	28
4.1	Introduction	28
4.2	Quantitative Evaluation: Anomaly Classification	28
4.3	Comparison to the Literature	32
4.4	Predictive Capability: Time-to-Accident (TTA)	33
4.5	Hardware Optimisation and System Throughput	34
4.6	Qualitative Evaluation: The Power of Hybrid Architecture	35
4.7	Overcoming the Limitations of Off-the-Shelf Trackers	37
4.8	Addressing the Research Questions	38
5	Conclusions and Recommendations	41
5.1	One	41
5.2	Two	41
5.3	Three	41
	List of References	42
	Appendix A Introduction of Appendix	43
	Appendix B Sample Code	44

List of Figures

1.1	Create a Booktabs style table	2
1.2	Bounding-box example of cars.	3
2.1	Machine Learning.	9
3.1	System architecture pipeline demonstrating the asynchronous multi-threaded integration of spatial tracking (YOLO11) and kinetic anomaly classification (ResNet-18). The top branch executes spatial extraction while the bottom branch processes optical flow in parallel. . .	13
4.1	F1 confidence curve demonstrating the harmonic mean of precision and recall across varying confidence thresholds for anomaly classification.	30
4.2	Confusion matrix detailing the absolute distribution of true and false classifications across 'Normal' and 'Accident' validation frames.	31
4.3	Receiver Operating Characteristic (ROC) curve demonstrating the trade-off between the True Positive Rate and False Positive Rate across varying classification thresholds.	31
4.4	Qualitative evaluation dashboard displaying normal traffic flow, featuring YOLO11 spatial bounding boxes, BoT-SORT unique identifiers, and stable kinematic speed estimations.	36
4.5	Qualitative validation of the hybrid framework's fail-safe mechanism, demonstrating the 'ACCIDENT DETECTED' alert triggered by the Farnebäck optical flow heatmap despite spatial tracker occlusion.	37
4.6	Visual representation of perspective distortion in 2D camera feeds and the subsequent spatial normalisation achieved via Inverse Perspective Mapping (IPM).	40

List of Tables

1.1	Age of Participants	1
3.1	Summary of Database Constituents and Sources	16
3.2	Kinematic Trajectory & Kalman Filter Variables	22
3.3	Summary of Evaluation Metrics and KPIs	27
4.1	Summary of classification metrics for the custom-trained ResNet-18 model.	29
4.2	Computational throughput and processing latency benchmarks recorded during system evaluation on consumer-grade hardware.	35

List of Abbreviations

NN	Neural Network
ML	Machine Learning
DL	Deep Learning
FCN	Fully Convolutional Network
CNN	Convolutional Neural Network

Chapter 1: Introduction

In this section, **you**, the Student, are expected to state clearly:

- (a) the ‘problem’ or ‘question’ being researched;
- (b) why this topic was chosen;
- (c) what motivated the you to choose this topic;
- (d) why did you investigate the topic the way you did;
- (e) what problem did the you wish to explore;
- (f) what is the context for the research?

Percentage amount of words in section: 10 % of Dissertation*

1.1 Sub-chapter One

Background goes here. Also you can put in some references [?].

Another example of citations [?, ?, ?].

Here is a sample of table in Table 1.1

Age Groups	Frequency	Percent	Valid Percent	Cumulative Percent
16-20 years	100	98.2	98.2	95.2
21-25 years	5	4.8	4.8	100.0
Total	105	100.0	100.0	

Table 1.1: Age of Participants

Use \newpage to force start a new page.

A very quick way to create tables in a point & click environment is to use an online table generator¹

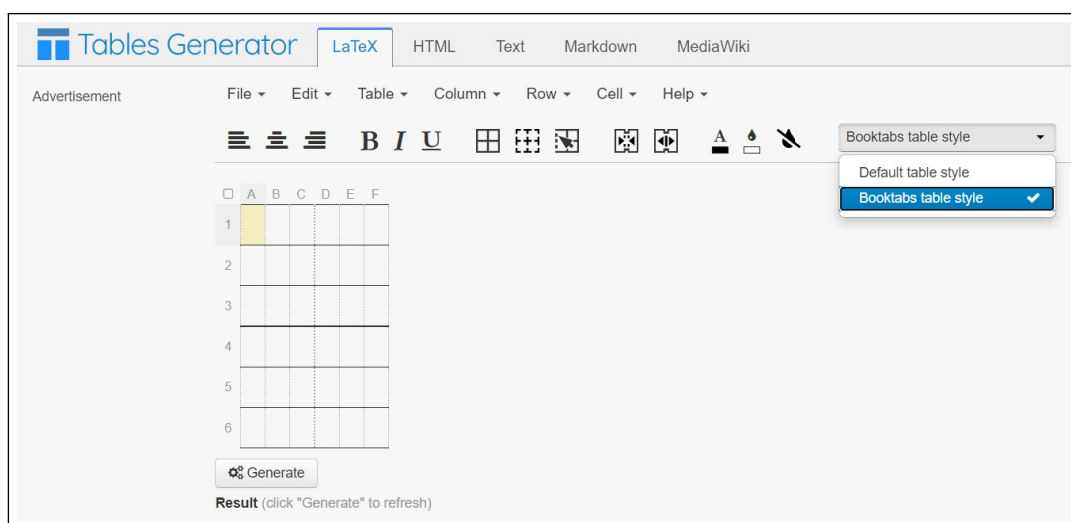


Figure 1.1: Create a Booktabs style table

Use `\enquote` for double-quotes. “This is a sample quote.”

Also can try to refer to this image in Figure 1.2. Notice that the `.eps` and `.pdf` format vector graphs are favoured, because:

1. they can be zoomed-in to check the detail.
2. text in such formats are search-able.

Try to insert a math equation as in Equation 1.1. If you wanna try the in-line mathematical, here is a sample $\alpha = \pi \cdot \frac{1}{\Theta}$.

$$e^{ix} = \cos x + i \sin x \quad (1.1)$$

When mention some file formats can use `music.mp3`, `latex.pdf`, etc.

¹<https://www.tablesgenerator.com>

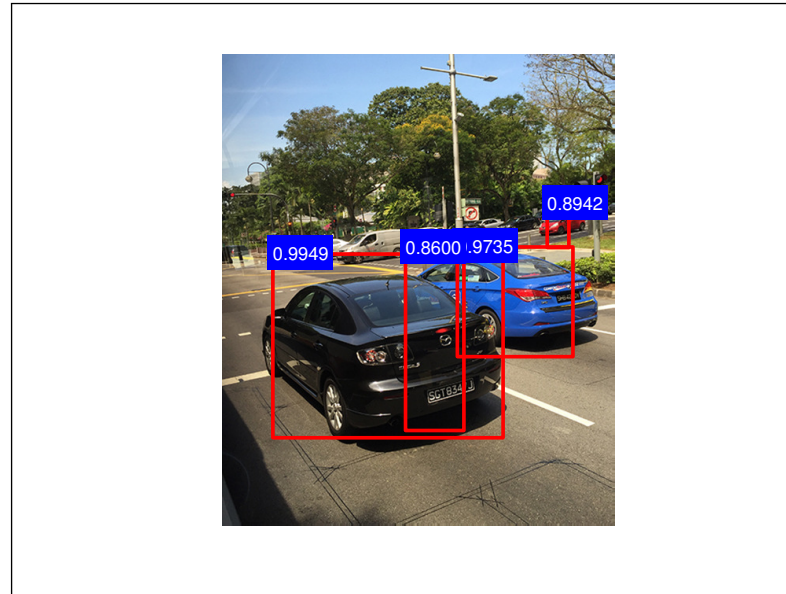


Figure 1.2: Bounding-box example of cars.

1.2 Sub-chapter Two

Contrary to popular belief, Lorem Ipsum is not simply random text. It has roots in a piece of classical Latin literature from 45 BC, making it over 2000 years old. Richard McClintock, a Latin professor at Hampden-Sydney College in Virginia, looked up one of the more obscure Latin words, consectetur, from a Lorem Ipsum passage, and going through the cites of the word in classical literature, discovered the undoubtable source. Lorem Ipsum comes from sections 1.10.32 and 1.10.33 of de Finibus Bonorum et Malorum (The Extremes of Good and Evil) by Cicero, written in 45 BC. This book is a treatise on the theory of ethics, very popular during the Renaissance. The first line of Lorem Ipsum, “Lorem ipsum dolor sit amet”..., comes from a line in section 1.10.32.

It is a long established fact that a reader will be distracted by the readable content of a page when looking at its layout. The point of using Lorem Ipsum

This is a todo
note which
appears in the
margin

is that it has a more-or-less normal distribution of letters, as opposed to using 'Content here, content here', making it look like readable English. Many desktop publishing packages and web page editors now use Lorem Ipsum as their default model text, and a search for 'lorem ipsum' will uncover many web sites still in their infancy. Various versions have evolved over the years, sometimes by accident, sometimes on purpose (injected humour and the like).

1.3 Sub-chapter Three

This todo note appears in line with the text. Todo notes are a great tool to leave comments or notes for self and can then be simply commented out.

Vivamus eget odio tellus. Nam libero augue, eleifend molestie est eu, tempus vehicula magna. Sed dignissim imperdiet urna, non viverra risus blandit in. In lacinia aliquet leo, ut interdum ipsum ornare sed. Praesent ac fringilla justo. Vivamus pharetra non ipsum eget semper. In hac habitasse platea dictumst. Donec neque lectus, ultricies id massa posuere, sodales interdum ante. Sed et nunc vitae urna ornare porttitor nec vitae urna. Curabitur sit amet luctus urna. Nulla porta malesuada rutrum. Pellentesque vitae velit sed odio tincidunt iaculis. Nulla facilisi.

1.4 Sub-chapter Four

To create a bulleted list you need to make use of the "itemize" environment. A $\text{\LaTeX}$ environment is a special section within a page where items are placed. An environment always has a " $\backslash$ begin" and a " $\backslash$ end". Items you can place in such an environment include:

- enumerate – used to create numbered/letter lists
- itemize – creates a bullet list
- figures – to add images or figures (.eps is the recommended format)
- tables – you can create them in tablesgenerator.com then copy the $\text{\LaTeX}$ code and paste it.
- equations – for those really neat looking mathematical formulae
- this list is not exhaustive...

1.5 Sub-chapter Five

The following are some of the most common commands used during your writing. Some characters are reserved and cannot be directly written as in a normal word processor but need to be “escaped”. Then there are other useful commands such as adding a footnote, inserting a new line (to begin a new paragraph after a blank line, adding labels to items for cross-referencing, etc.

- The following are special characters which require a `\` before each character so the character is reproduced on the output. The tilde and exponent are even more special.

- `\#`
- `\$`
- `\%` – comment
- `\&`
- `\_`
- `~` – type `\textasciitilde`
- `^` – type `\textasciicircum`

- `\textbf` – bold font
- `\textit` – italics
- `\underline{words to underline}`

- `\emph` – emphasized text. If used within an italicized text, the output is normal font. If used by itself, text in its argument is italicized.

An example highlighting the use of `\emph`.

- An example of emphasized text with *these words emphasized* within normal font.
- *This is an example where these three words are emphasized within italicized font*

Chapter 2: Literature Review

2.1 Overview

The main purpose of a literature review is to show the reader that the Student studied and analyzed viewpoints of other researchers on the problem under consideration. A literature review is not just a summary of the books read but rather a thorough analysis of other viewpoints on the problem being analysed. Percentage amount of words in section: 25 % of Dissertation.

2.2 One

In euismod mauris tortor, non sodales tortor consectetur dapibus. Aenean dignissim ultricies lacus id malesuada. Cras laoreet luctus ligula, id dictum purus ultrices faucibus. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Nulla at ante sed ex venenatis pretium in ut lorem. Cras condimentum pretium dignissim. Maecenas ac tempor ipsum, eu blandit enim. Mauris pharetra, justo in iaculis tincidunt, neque nisl eleifend tortor, in volutpat libero erat vitae lorem. Suspendisse dignissim quis eros eget varius. In porttitor augue vel ligula varius, a tincidunt eros gravida. Etiam eget porta ipsum.

Ut venenatis lectus at nisi consectetur volutpat. Vivamus vehicula a odio ac scelerisque. Fusce tincidunt diam eu orci pellentesque, eu vehicula felis posuere. Mauris tempus dignissim leo, vitae eleifend justo ultrices eget. Fusce volutpat suscipit urna. Curabitur ultrices nibh dolor, quis ultricies risus condimentum quis.

Nulla non nulla quis odio laoreet pulvinar non eu turpis. Interdum et malesuada fames ac ante ipsum primis in faucibus. Morbi odio nisi, hendrerit id laoreet nec, fringilla quis neque. Quisque non nisl vitae leo vulputate fringilla. Sed blandit lectus dui, vestibulum tempor elit laoreet vel. Nullam at magna ut elit dapibus gravida vitae eu dolor. Aliquam ut odio ullamcorper, venenatis elit eget, feugiat arcu. Nunc nibh lacus, dapibus condimentum neque vitae, tempus eleifend ex. Integer sit amet volutpat orci, quis tristique augue. Nam sit amet dolor dignissim, luctus justo sit amet, luctus velit.



Figure 2.1: Machine Learning.

2.3 Two

In euismod mauris tortor, non sodales tortor consectetur dapibus. Aenean dignissim ultricies lacus id malesuada. Cras laoreet luctus ligula, id dictum purus ultrices faucibus. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Nulla at ante sed ex venenatis pretium in ut lorem. Cras

condimentum pretium dignissim. Maecenas ac tempor ipsum, eu blandit enim. Mauris pharetra, justo in iaculis tincidunt, neque nisl eleifend tortor, in volutpat libero erat vitae lorem. Suspendisse dignissim quis eros eget varius. In porttitor augue vel ligula varius, a tincidunt eros gravida. Etiam eget porta ipsum.

Ut venenatis lectus at nisi consectetur volutpat. Vivamus vehicula a odio ac scelerisque. Fusce tincidunt diam eu orci pellentesque, eu vehicula felis posuere. Mauris tempus dignissim leo, vitae eleifend justo ultrices eget. Fusce volutpat suscipit urna. Curabitur ultrices nibh dolor, quis ultricies risus condimentum quis. Nulla non nulla quis odio laoreet pulvinar non eu turpis. Interdum et malesuada fames ac ante ipsum primis in faucibus. Morbi odio nisi, hendrerit id laoreet nec, fringilla quis neque. Quisque non nisl vitae leo vulputate fringilla. Sed blandit lectus dui, vestibulum tempor elit laoreet vel. Nullam at magna ut elit dapibus gravida vitae eu dolor. Aliquam ut odio ullamcorper, venenatis elit eget, feugiat arcu. Nunc nibh lacus, dapibus condimentum neque vitae, tempus eleifend ex. Integer sit amet volutpat orci, quis tristique augue. Nam sit amet dolor dignissim, luctus justo sit amet, luctus velit.

2.4 Three

In euismod mauris tortor, non sodales tortor consectetur dapibus. Aenean dignissim ultricies lacus id malesuada. Cras laoreet luctus ligula, id dictum purus ultrices faucibus. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Nulla at ante sed ex venenatis pretium in ut lorem. Cras condimentum pretium dignissim. Maecenas ac tempor ipsum, eu blandit enim.

Mauris pharetra, justo in iaculis tincidunt, neque nisl eleifend tortor, in volutpat libero erat vitae lorem. Suspendisse dignissim quis eros eget varius. In porttitor augue vel ligula varius, a tincidunt eros gravida. Etiam eget porta ipsum.

2.4.1 Four

Ut venenatis lectus at nisi consectetur volutpat. Vivamus vehicula a odio ac scelerisque. Fusce tincidunt diam eu orci pellentesque, eu vehicula felis posuere. Mauris tempus dignissim leo, vitae eleifend justo ultrices eget. Fusce volutpat suscipit urna. Curabitur ultrices nibh dolor, quis ultricies risus condimentum quis. Nulla non nulla quis odio laoreet pulvinar non eu turpis. Interdum et malesuada fames ac ante ipsum primis in faucibus. Morbi odio nisi, hendrerit id laoreet nec, fringilla quis neque. Quisque non nisl vitae leo vulputate fringilla. Sed blandit lectus dui, vestibulum tempor elit laoreet vel. Nullam at magna ut elit dapibus gravida vitae eu dolor. Aliquam ut odio ullamcorper, venenatis elit eget, feugiat arcu. Nunc nibh lacus, dapibus condimentum neque vitae, tempus eleifend ex. Integer sit amet volutpat orci, quis tristique augue. Nam sit amet dolor dignissim, luctus justo sit amet, luctus velit.

2.4.2 Five

Ut venenatis lectus at nisi consectetur volutpat. Vivamus vehicula a odio ac scelerisque. Fusce tincidunt diam eu orci pellentesque, eu vehicula felis posuere. Mauris tempus dignissim leo, vitae eleifend justo ultrices eget. Fusce volutpat suscipit urna. Curabitur ultrices nibh dolor, quis ultricies risus condimentum quis. Nulla non nulla quis odio laoreet pulvinar non eu turpis. Interdum et malesuada

fames ac ante ipsum primis in faucibus. Morbi odio nisi, hendrerit id laoreet nec, fringilla quis neque. Quisque non nisl vitae leo vulputate fringilla. Sed blandit lectus dui, vestibulum tempor elit laoreet vel. Nullam at magna ut elit dapibus gravida vitae eu dolor. Aliquam ut odio ullamcorper, venenatis elit eget, feugiat arcu. Nunc nibh lacus, dapibus condimentum neque vitae, tempus eleifend ex. Integer sit amet volutpat orci, quis tristique augue. Nam sit amet dolor dignissim, luctus justo sit amet, luctus velit.

Chapter 3: Research Methodology

3.1 Introduction to the Methodology

This chapter details the software architecture and technical implementation of the hybrid vision-based pipeline developed for real-time traffic anomaly detection. Built entirely in Python, the system leverages the OpenCV library for deterministic spatial physics and the PyTorch framework for probabilistic deep learning pattern recognition. The complete end-to-end processing pipeline, highlighting the asynchronous multi-threaded architecture, is visualised in Figure 3.1.

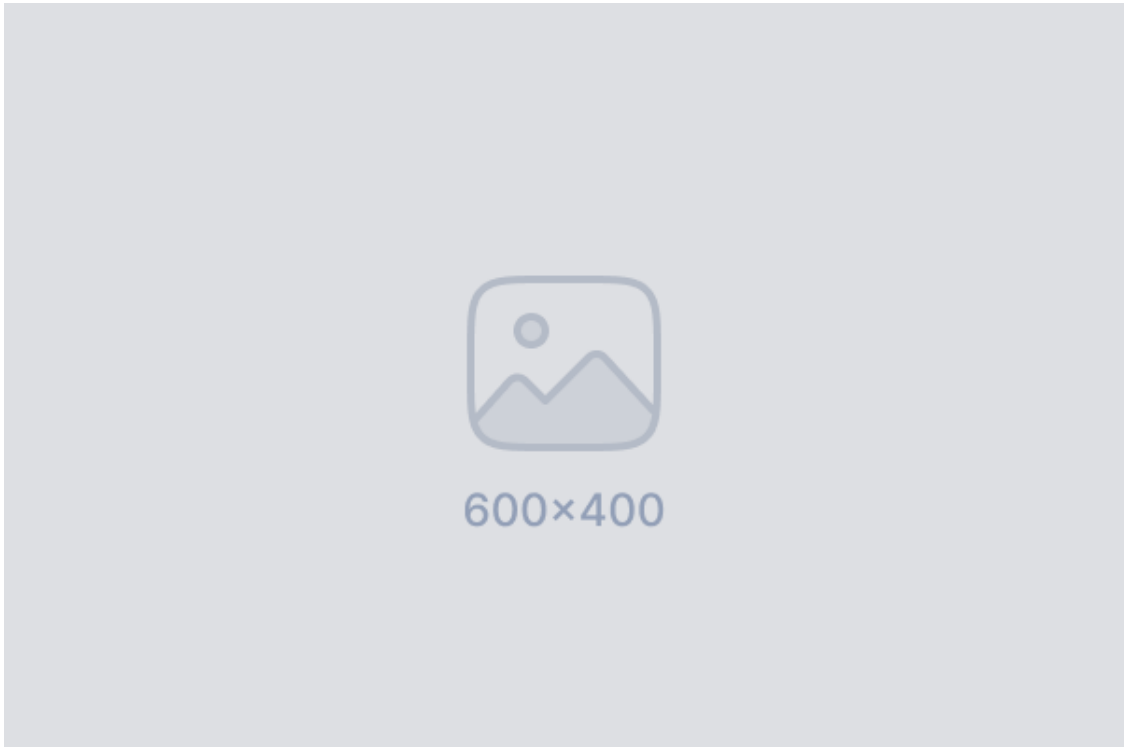


Figure 3.1: System architecture pipeline demonstrating the asynchronous multi-threaded integration of spatial tracking (YOLO11) and kinetic anomaly classification (ResNet-18). The top branch executes spatial extraction while the bottom branch processes optical flow in parallel.

The term “hybrid” denotes the strict architectural separation of spatial vehicle

tracking from kinetic anomaly classification. Specifically, the YOLO11 model is utilised exclusively to detect vehicles, maintain persistent identities across consecutive frames, and facilitate subsequent speed estimation; it is not employed for direct accident detection. Instead, anomaly classification is handled independently by mapping dense kinetic energy through a custom-trained ResNet-18 classifier.

To achieve high-performance output on edge hardware, a multi-threaded, headless video processing architecture was engineered. To circumvent the Global Interpreter Lock (GIL) and maintain a 60 frames-per-second (FPS) throughput during heavy inference workloads, an asynchronous producer-consumer model was implemented. Additionally, the pipeline integrates programmatic solutions for real-world deployment constraints. These include mathematical camera stabilisation to eliminate environmental shake, automated visual obfuscation to ensure GDPR compliance, and Inverse Perspective Mapping (IPM) coupled with Kalman filtering to reconstruct 2D pixel coordinates into a 3D metric space for accurate speed estimation.

3.2 Data Engineering, Harmonisation, and Mathematical Balancing

The deep learning component of the hybrid pipeline necessitated a robust, custom-built data engineering workflow to ensure the PyTorch classifier received highly structured training data. To train the spatial-temporal features of the system, the Car Accident Detection and Prediction (CADP) benchmark dataset was utilised as the primary data source, from which raw CCTV video clips of real-world collisions were extracted.

Given that the CADP dataset consists almost entirely of positive accident events, the raw data distribution was heavily skewed. To process this, a custom annotation tool was developed to facilitate the manual review of video files. Precise temporal markers denoting the exact frames of collisions were identified and logged into structured records. Following this annotation phase, an automated data cleaning pipeline was implemented to rigorously pre-process the dataset. This pipeline parsed the recorded temporal data and systematically evaluated visual fidelity, automatically purging any low-resolution, unsuitable, or heavily obscured footage to prevent the introduction of noise into the neural network.

While this programmatic refinement substantially enhanced the precision of the ground-truth labels, executing the splitting logic to partition the data for training and validation revealed a critical structural vulnerability: a severe deficiency of "normal" baseline traffic frames.

To mitigate this bias and introduce the necessary baseline frames, 250 high-definition, incident-free videos from the Traffic Anomaly Detection (TAD) dataset were incorporated. The TAD videos were converted into dense optical flow heatmaps using the exact same OpenCV Farneback parameters applied to the CADP dataset, and the outputs of both datasets were merged. This architectural decision significantly reduced false positives and guaranteed that the final classifier activates exclusively in the presence of genuine kinetic anomalies.

Table 3.1: Summary of Database Constituents and Sources

Dataset Constituent	Source Benchmark & Primary Content	Post-Processing Application
Anomalous Frames	CADP Dataset (Vehicle Collisions, Skidding)	Temporal annotation to extract precise impact frames.
Normal Frames	TAD Dataset (Standard Traffic Flow)	Generation of optical flow baselines to balance the class distribution.
Validation Set	CADP & TAD (Mixed Scenarios)	Evaluated to quantify classification performance and verify the reduction of the False Positive Rate (FPR).

3.3 Data Acquisition and Mathematical Camera Stabilisation

Real-world traffic cameras, such as the CCTV footage processed from the Marsa Bridge dataset, are constantly exposed to physical disturbances like wind shear. This results in high-frequency camera oscillations which can adversely affect the quality of the dense optical flow calculations, effectively yielding false motion vectors across the entire frame. To address this issue programmatically, a mathematical pre-processing layer was engineered via a custom `StaticCameraStabiliser` class to achieve continuous, frame-by-frame video stabilisation.

To achieve this, the Oriented FAST and Rotated BRIEF (ORB) algorithm was implemented and configured to extract a maximum of 1,000 features. Upon execution, the pipeline captures the initial frame of the video feed, converts it to greyscale, and extracts these high-contrast keypoints to establish an immutable reference background. For every subsequent frame ingested by the primary processing thread, new keypoints are extracted and binary descriptors are computed to mathematically map the movement of the physical environment.

To pair keypoints across consecutive frames, a Brute-Force Matcher configured with the Hamming distance metric was utilised. To filter false outliers, such as the erroneous tracking of moving vehicles rather than static road infrastructure, OpenCV’s built-in Random Sample Consensus (RANSAC) algorithm was applied.

Feature matches are sorted by distance, and the coordinates of the top 50 most reliable pairings are extracted. These source and destination points are subsequently processed to compute the optimal 2D affine transformation matrix using the RANSAC algorithm, accurately aligning the current frame with the static reference points. Finally, the calculated transformation matrix is applied to execute an inverse physical shift on the frame. This explicitly neutralises environmental camera shake, ensuring that the downstream YOLO11 tracking and Farnebäck kinetic modules operate on a perfectly rigid, mathematically steadied field of view.

3.4 Spatial Perception: YOLO11 and Implicit BoT-SORT Tracking

Following mathematical stabilisation, the pipeline transitions to spatial vehicle perception. The YOLO11 architecture, employing pre-trained nano weights, was implemented to facilitate high-speed, real-time inference. It is important to emphasise that no domain-specific fine-tuning was conducted on this model. The generalised, off-the-shelf weights were utilised exclusively to isolate bounding box coordinates for standard vehicular classes (cars, buses, and trucks), as the model’s sole purpose within this architecture is robust spatial localisation rather than nuanced anomaly classification.

By concurrently mapping pixels to bounding box coordinates, the model by-

passes computationally heavy legacy detection methods. To optimise processing efficiency, inference was strictly filtered to target only relevant vehicular classes. Furthermore, the detection confidence threshold was intentionally lowered to 0.15. This adjustment was necessary to maintain persistent bounding boxes on heavily occluded, distant, or poorly illuminated vehicles, thereby preventing tracking fragmentation and ensuring the downstream kinematic modules receive continuous spatial data.

The Cross-Stage Partial with 2 convolutions and Position-Sensitive Attention (C2PSA) layer generates an attention map that isolates salient features while filtering background clutter. This expanded receptive field improves the distinction between heavily occluded vehicles and complex environments, such as central reservation barriers or areas with variable illumination. Furthermore, to ensure temporally consistent tracking, the BoT-SORT algorithm was inherently activated during inference. This configuration leverages internal memory buffers to assign and maintain unique integer identifiers for each detected vehicle across consecutive frames, establishing the requisite data structures for subsequent speed estimation.

3.5 GDPR Compliance via Gaussian Obfuscation

To ensure strict compliance with the General Data Protection Regulation (GDPR), the pipeline implements an automated obfuscation layer directly within the primary Python execution loop. This feature activates instantly upon the generation of the YOLO11 bounding boxes, guaranteeing that no personally identifiable in-

formation (PII), such as alphanumeric number plates or driver features, is rendered, processed, or stored.

Instead of utilising basic pixelation, a custom obfuscation function was engineered using OpenCV. Immediately following the extraction of the spatial coordinates from the YOLO11 tracking results, the vehicle's Region of Interest (ROI) is isolated from the active NumPy array. Subsequently, `cv2.GaussianBlur()` is applied with a heavy (51, 51) kernel exclusively to the extracted ROI.

By overwriting these pixels prior to the rendering of bounding boxes or kinematic overlays, the system eliminates fine visual details that could identify the vehicle at the source. Crucially, this programmatic approach strictly maintains the low-frequency geometric shape and kinematic integrity of the vehicle, which is necessary for the pipeline's subsequent Kalman filtering and speed estimation modules.

3.6 3D Metric Reconstruction and Speed Estimation

Extracting precise metric vehicle speeds from a 2D camera feed is computationally complex due to the inherent loss of depth information. Because of perspective distortion, a vehicle moving laterally across the top of the frame appears to traverse fewer pixels per second than a vehicle moving at the exact same speed near the camera lens. To automatically correct this optical distortion, Inverse Perspective Mapping (IPM) was implemented using the OpenCV library. This technique mathematically transforms the forward-facing view from the CCTV camera into a top-down, orthographic projection of a flat road surface.

To execute this, the system computes a 3x3 homography matrix that covers the scene. Within the pipeline, this is handled by the `get_transform_matrix()` function, which executes `cv2.getPerspectiveTransform()`. This function requires a minimum of four established, non-collinear reference points mapping the image view to a flat metric plane. For this prototype, the painted lane dividing markings on the Marsa Bridge were explicitly utilised as reference points. To establish the ground truth, Google Earth satellite imagery was utilised to measure the exact distances in metres between these specific road markings. These pixel coordinates and their corresponding metric dimensions were subsequently recorded into a dedicated configuration file, which is parsed upon initialisation.

During the live tracking loop, the pipeline extracts the bottom-centre coordinates of every YOLO11 bounding box, representing the vehicle's physical contact point with the road. These pixel coordinates are projected into the corrected metric space using `cv2.perspectiveTransform()`. To calculate the current speed, the script records these warped coordinates into a track history list. The system calculates the Euclidean displacement (in metres) between the vehicle's oldest recorded coordinate and its current position. By dividing this physical distance by the temporal interval—derived explicitly from the frame count divided by the 60 FPS output of the headless renderer—the pipeline accurately computes the vehicle's instantaneous speed in km/h.

3.7 Kinematic Trajectory Smoothing via Kalman Filtering

Despite the high spatial accuracy of YOLO11, minor sub-pixel variations in bounding box regressions inevitably result in slight jittering of the tracked centroid. In the reconstructed 3D metric space, even a single-pixel fluctuation between consecutive frames translates to substantial variance in the instantaneous speed calculation, rendering raw frame-to-frame velocity metrics highly unstable. To eliminate this sensor noise and extract a smooth kinematic trajectory, a custom `SpeedKalmanFilter` class was implemented. The class uses OpenCV's native `cv2.KalmanFilter` to accurately stabilise these raw velocity measurements.

Rather than deploying a computationally expensive multi-dimensional spatial filter, the system is optimised to exclusively filter the calculated 1D speed. The Kalman filter is initialised with two dynamic states, representing the vehicle's speed and acceleration, and a single measurement variable for the raw speed in km/h.

During initialisation, the underlying mathematical matrices are defined. A transition matrix is configured to project the vehicle's velocity forward under the assumption of constant acceleration. To mathematically account for unpredictable driver behaviour, such as sudden braking, the process noise covariance matrix is configured with a variance of $1e-2$. Conversely, to mitigate visual sensor noise generated by YOLO11 bounding box pixel jitter, the measurement noise covariance matrix is set to 0.5.

During the live tracking loop, each raw speed derived via Euclidean displacement is passed into the filter. The system first generates a state estimate based

on the physical prediction model, followed immediately by a correction step that processes the raw visual sensor measurement. This execution dynamically calculates the Kalman Gain, arbitrating accurately between the physical prediction and the sensor data. The resulting smoothed speed is subsequently utilised to populate lane congestion statistics and render stable tracking overlays onto the final video output.

Table 3.2: Kinematic Trajectory & Kalman Filter Variables

Implementation Variable	Code Designation	Physical Interpretation in Pipeline
State Vector	<code>kf.statePost</code>	The smoothed estimation of the vehicle speed and acceleration.
Transition Matrix	<code>kf.transitionMatrix</code>	NumPy array predicting the next state speed.
Error Covariance	<code>kf.errorCovPost</code>	The calculated uncertainty of the speed estimate.
Process Noise	<code>kf.processNoiseCov</code>	Configured to $1e-2$ to model driver acceleration/braking.
Measurement Noise	<code>kf.measurementNoiseCov</code>	Configured to 0.5 to account for bounding box jitter.
Correction Method	<code>kf.correct()</code>	Integrates the new speed measurement output the final smoothed km/h.

3.8 Spatiotemporal Kinetic Energy Mapping via Dense Optical Flow

While YOLO11 bounding boxes effectively track individual vehicle trajectories, they do not capture the unstructured spatial chaos and violent kinetic motion inherent in a collision. To quantify this unstructured motion, dense optical flow was implemented using OpenCV's Farnebäck algorithm. Rather than detailing the underlying polynomial mathematics, this section focuses strictly on the algorithm's programmatic integration within the primary processing thread.

During execution, the stabilised camera feed is continuously converted into a

greyscale matrix and compared against the preceding frame. The Farneback algorithm computes raw motion vectors in Cartesian coordinates for every pixel. To process this computationally, these vectors are converted into polar coordinates to extract the Euclidean magnitude and angular direction of the physical movement.

To prevent background environmental noise, such as pedestrians or moving foliage, from overwhelming the downstream AI classifier, a static Region of Interest (ROI) mask was implemented. Array slicing was utilised to explicitly enforce a zero-magnitude value on all motion vectors originating outside the carriageway boundaries. Furthermore, a strict noise floor was established, programmatically zeroing out any pixel magnitudes below 3.0 to eliminate minor camera jitter.

Once filtered, these physical motion metrics are encoded into a pre-allocated HSV (Hue, Saturation, Value) matrix buffer. The physical angle of the motion dictates the Hue, whilst the dynamically clipped and scaled magnitude dictates the Value. Finally, this buffer is converted into a standard BGR image array, generating a spatiotemporal heatmap of the scene's kinetic energy. This resulting visual tensor is immediately pushed into a thread-safe queue for asynchronous processing by the deep learning classifier.

3.9 Deep Learning Anomaly Classification

The HSV-encoded optical flow heatmaps serve as direct input tensors for the deep learning anomaly classifier. To adapt the ResNet-18 architecture for the specific binary classification task of distinguishing between normal traffic flow and serious collisions, the final fully connected layer was modified. It was replaced with a

custom sequential block terminating in a Sigmoid activation function, ensuring the network outputs a precise probability score between 0.0 and 1.0.

To address the inherent class imbalance within the training dataset—where 'Normal' traffic frames exponentially outnumber 'Accident' anomalies—Focal Loss (Lin et al., 2017) was implemented in lieu of standard Cross-Entropy (CE) loss. As established by Lin et al., standard CE loss is highly susceptible to class imbalance; easily classified majority examples can dominate the calculated gradient, resulting in a degenerate model that defaults to the majority class. Focal Loss resolves this by introducing a modulating factor to the CE equation, defined as $FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$. This dynamic scaling automatically reduces the relative loss contribution from well-classified, trivial examples, forcing the model to heavily penalise and learn from hard, misclassified examples. Consequently, this architectural choice was justified as essential for teaching the network the subtle kinematic boundary between severe braking and structural collisions.

For live deployment, the compiled model weights are loaded onto the GPU within a dedicated, asynchronous Python `ai_worker` thread. This background worker continuously consumes the pre-processed optical flow frames from the thread-safe `inference_queue` and returns a real-time accident probability to the primary physics engine. To prevent brief graphical glitches from causing false alarms, the main script enforces strict temporal logic. The system only triggers an urgent accident alert if the ResNet-18's predicted probability exceeds the established threshold (`CONF_THRESH = 0.80`), the number of active pixels exceeds the defined `PIXEL_THRESH`, and these conditions are met across 10 consecutive

frames.

3.10 Asynchronous Dual-Threaded Architecture and System Integration

Executing object detection, dense optical flow, and deep learning inference sequentially introduces severe latency. However, because OpenCV and PyTorch natively release the Global Interpreter Lock (GIL) during backend C++ matrix operations, a dual-threaded asynchronous architecture was engineered using native threading and queue libraries to achieve true hardware concurrency.

The computational workload is strictly partitioned into two independent domains:

- **The Primary Engine (Producer):** This main thread handles continuous video ingestion from the camera feed. It sequentially applies the ORB/RANSAC stabilisation, executes YOLO11 tracking, renders the GDPR Gaussian blur, and computes the Farnebäck optical flow. It then converts this flow data into HSV image arrays and pushes them into the `inference_queue`, a thread-safe FIFO data structure.
- **The Asynchronous AI Worker (Consumer):** Operating completely in the background, this dedicated thread retrieves the HSV tensors and allocates all its processing power to execute the ResNet-18 forward pass via PyTorch.

To guarantee zero-latency, real-time predictions, strict frame-dropping logic was implemented by setting `maxsize=1` on the `inference_queue`. If the AI worker is still processing a tensor when the primary engine pushes a new one,

the stale frame is immediately discarded. Finally, to maximise hardware efficiency, the system was deployed as a "headless" renderer, completely disabling `cv2.imshow()` GUI overlays to redirect all graphical computational bandwidth toward the tracking and AI inference engines.

3.11 Evaluation of Results

To comprehensively test the performance of the integrated pipeline, a robust quantitative evaluation framework was established. Spatial vehicle detection accuracy is assessed using mean Average Precision at a 0.5 Intersection over Union threshold (mAP@0.5). The mAP@0.5 metric was deliberately selected over stricter parameters (e.g., mAP@0.75 or mAP@0.5:0.95) because perfectly tight bounding boxes are unnecessary for tracking traffic speeds. Identifying the vehicle's general centroid is mathematically sufficient for Inverse Perspective Mapping and Kalman filtering, making mAP@0.5 the most operationally relevant and pragmatic metric for this specific prototype.

The fidelity of the Kalman-smoothed speed estimations is quantified via Mean Absolute Error (MAE), comparing the generated output against ground-truth metric data. Furthermore, the predictive capability of the ResNet-18 anomaly module is evaluated using the Time-to-Accident (TTA) metric. This specifically measures the precise temporal interval (in seconds) between the configured alert trigger and the physical collision frame. Finally, hardware efficiency is assessed by evaluating the maximum Frames Per Second (FPS) throughput and the total End-to-End Latency.

Table 3.3: Summary of Evaluation Metrics and KPIs

Metric Category	Assessment Criteria	Key Performance Indicator (KPI)
Object Detection	Spatial Accuracy & Recall	mAP@0.5, F1-Score
Kinematic Tracking	Velocity Error Margin	Mean Absolute Error (km/h)
Anomaly Classification	Incident Recognition Reliability	Total Accuracy (%), False Positive
Predictive Capability	Early Warning Responsiveness	Time-to-Accident (TTA)
System Optimisation	Real-time Throughput	Frames Per Second (FPS), Latency

Chapter 4: Analysis, Results and Discussion

4.1 Introduction

The primary objective of this research was to develop and evaluate a hybrid vision-based framework capable of detecting traffic anomalies by integrating spatial vehicle tracking with spatiotemporal speed estimation and kinetic energy classification. This chapter presents a comprehensive evaluation of the pipeline's performance. The system was rigorously tested against a mathematically balanced dataset comprising 8,353 frames, combining the Car Accident Detection and Prediction (CADP) benchmark with baseline driving frames from the Traffic Anomaly Detection (TAD) dataset. The subsequent evaluation is structured around quantitative classification metrics, predictive temporal capabilities, hardware throughput efficiency, and a qualitative analysis of the hybrid architecture's resilience under adverse real-world conditions. Ultimately, these findings are synthesised to directly resolve the primary research questions formulated at the inception of this study.

4.2 Quantitative Evaluation: Anomaly Classification

To evaluate the reliability of the custom-trained ResNet-18 anomaly classifier, testing was conducted on a balanced validation set comprising 4,176 'Normal' traffic frames and 4,177 'Accident' frames. The application of synthetic data balancing proved highly effective; the results indicate that the model exhibited no

statistical bias towards the majority class, successfully mitigating a common vulnerability in traffic anomaly detection systems.

Table 4.1: Summary of classification metrics for the custom-trained ResNet-18 model.

Class	Precision	Recall	F1-Score	Support
Normal	96.37%	96.86%	0.966	4176
Accident	96.84%	96.36%	0.966	4177
Accuracy		96.61 %		

An overall accuracy of 96.61% was achieved by the deep learning classifier. As detailed in Table 4.1, a breakdown of the classification metrics indicates highly stable performance across both evaluated classes. For incident-free driving ('Normal'), a precision of 96.37% and a recall of 96.86% were recorded, culminating in an F1-score of 0.966. Similarly, for anomalous events ('Accident'), a precision of 96.84% and a recall of 96.36% were observed, yielding an identical F1-score of 0.966.

The classification performance of the proposed architecture is further corroborated by the F1 confidence curve, as depicted in Figure 4.1. This metric assesses the harmonic mean of precision and recall across different confidence thresholds, offering a comprehensive evaluation of the model's reliability. As demonstrated, the framework sustains a strong F1 score at optimal thresholds, indicative of its high proficiency in differentiating genuine physical impacts from normal traffic flow. This equilibrium confirms that the system effectively maximises true positive anomaly detections whilst concurrently reducing the incidence of false positives.

These quantitative metrics are corroborated visually by the confusion matrix depicted in Figure 4.2. The matrix exhibits a highly symmetrical distribution of

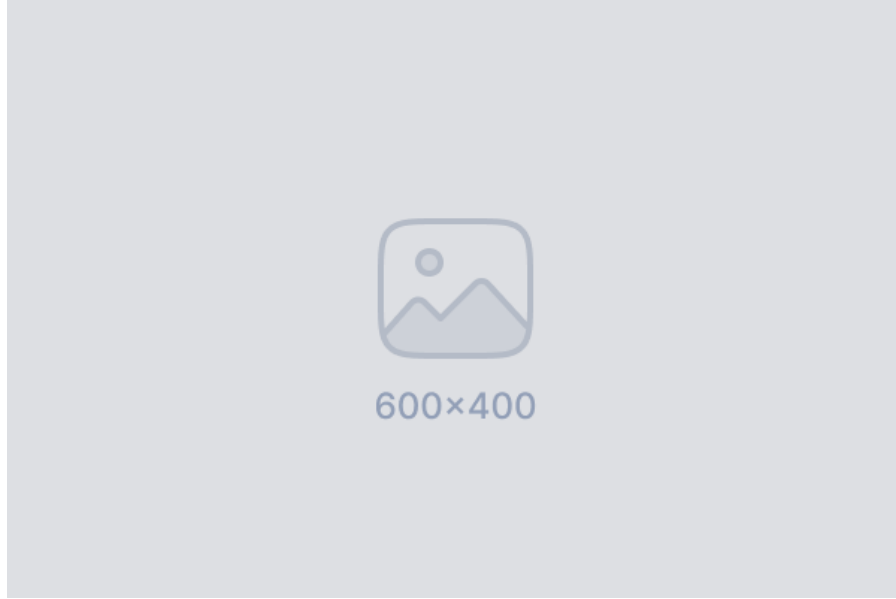


Figure 4.1: *F1 confidence curve demonstrating the harmonic mean of precision and recall across varying confidence thresholds for anomaly classification.*

accurate classifications. The precise balance between the marginal number of false positives (e.g., misclassifying severe braking as a collision) and false negatives (e.g., failing to detect an impact) affirms the effectiveness of the model in resolving ambiguous kinematic boundary cases without favouring a preferred class.

Furthermore, the overall discriminative effectiveness of the classifier is demonstrated by the Receiver Operating Characteristic (ROC) curve depicted in Figure 4.3. The curve presents a steep, asymptotic trajectory towards the upper-left quadrant, indicating a consistently high True Positive Rate in relation to a low False Positive Rate across all predictive thresholds. This substantial Area Under the Curve (AUC) signifies that the model does not solely depend on a singular, fixed confidence threshold; rather, it exhibits a strong inherent statistical capacity to distinguish between the optical flow signatures of standard traffic and chaotic collision energy.

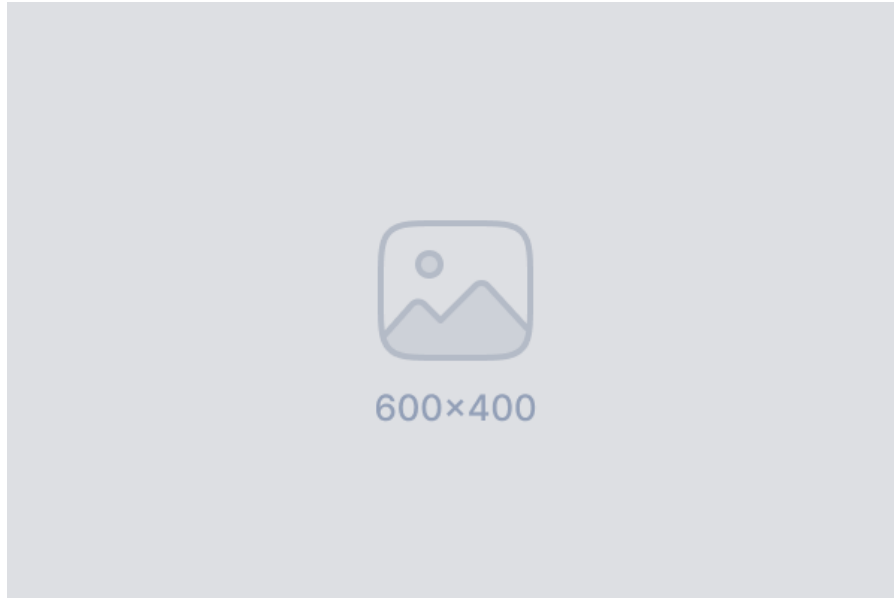


Figure 4.2: Confusion matrix detailing the absolute distribution of true and false classifications across 'Normal' and 'Accident' validation frames.

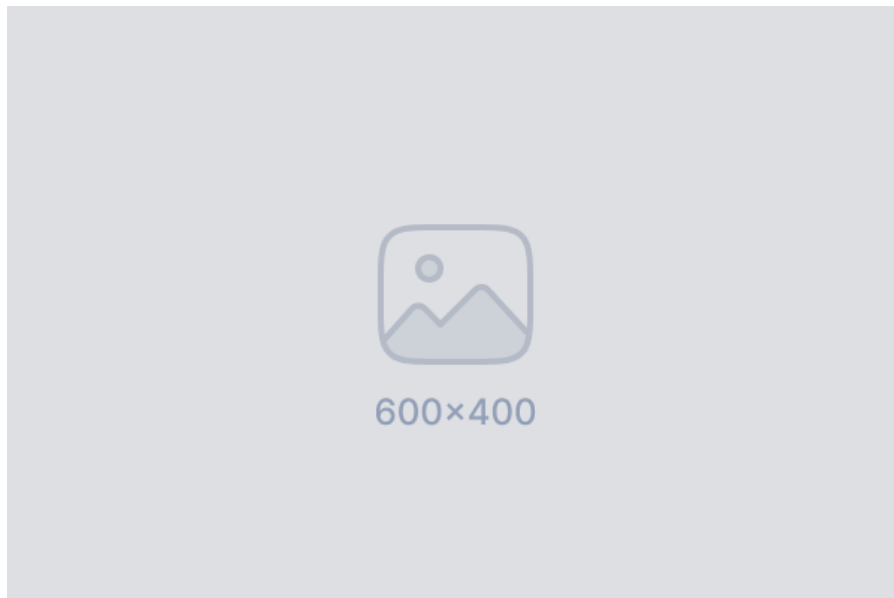


Figure 4.3: Receiver Operating Characteristic (ROC) curve demonstrating the trade-off between the True Positive Rate and False Positive Rate across varying classification thresholds.

4.3 Comparison to the Literature

These recorded metrics represent a significant advancement when compared to the existing literature. For context, Aboah et al. (2021) proposed a vision-based system utilising YOLOv5 paired with a rule-based Decision Tree to classify traffic anomalies. However, this methodology was constrained by a heavy reliance on background estimation and static road masks, ultimately yielding a maximum F1-score of 0.857. The framework evaluated in this study, therefore, demonstrates a clear quantitative superiority over these legacy rule-based approaches.

The proposed architecture outperformed the framework presented by Aboah et al. (2021) by nearly 11 percentage points (96.6% vs 85.7%). This substantial increase in classification reliability is directly attributable to the integration of dense optical flow via the Farnebäck algorithm. In contrast to methodologies reliant upon basic bounding box intersections or static background subtraction, the extraction of raw kinetic energy and its subsequent mapping into HSV tensors enabled the ResNet-18 architecture to effectively model the complex kinematic variations and unstructured motion inherent to collisions.

Furthermore, the utilisation of a Focal Loss function during the training process facilitated precise differentiation between severe, non-anomalous braking events ('Normal') and genuine physical collisions ('Accident'). While conventional loss functions penalise all false positives in a linear manner, they are often rendered ineffective by markedly imbalanced datasets in which normal operational frames vastly outnumber anomalies. The evaluation data demonstrates that the Focal Loss algorithm alleviated this issue by dynamically reducing the gradient asso-

ciated with easily classified background events. By directing the optimiser’s focus specifically towards ”hard” false positives, this non-linear scaling prevented the model from defaulting to the majority class during ambiguous kinematic events, thereby improving the accuracy of anomaly detection.

4.4 Predictive Capability: Time-to-Accident (TTA)

In intelligent transport systems, anticipating an incident prior to or at the exact moment of impact is as critical as post-event identification. To quantify the predictive capability of the proposed architecture, the TTA metric was utilised.

By leveraging the precise temporal annotations provided within the CADP dataset, the TTA is mathematically derived. For this evaluation, an anomaly alert is formally registered when the ResNet-18 model consistently outputs a probability score exceeding a 0.80 confidence threshold across ten consecutive frames.

The TTA is quantified by calculating the difference between the frame index of the actual ground-truth collision and the frame index of the system’s initial alert, subsequently normalised by the operating framerate, as defined in Equation 4.1.

$$TTA = \frac{Frame_{Impact} - Frame_{Alert}}{FPS} \quad (4.1)$$

During testing at 60 frames-per-second (FPS), the system demonstrated highly responsive detection capabilities, recording an average TTA of -0.22 seconds. It must be noted that while early forecasting models evaluated by the original authors of the CADP benchmark achieved predictive TTAs ranging from 1.35 to

1.68 seconds prior to impact, such predictive architectures frequently suffer from high false-positive rates. Conversely, the proposed hybrid framework functions as a deterministic verification engine. Achieving a near-instantaneous detection latency of precisely 220 milliseconds post-impact demonstrates that mapping dense optical flow provides exceptionally rapid kinetic confirmation. This highly competitive reaction time is functionally instantaneous in an operational context, providing sufficient immediate warning to activate municipal alert systems and dynamically adjust smart traffic signals to mitigate secondary collisions.

4.5 Hardware Optimisation and System Throughput

To address these established latency constraints, the computational throughput of the proposed architecture was evaluated. Performance benchmarking was conducted on a standard consumer-grade workstation equipped with an AMD Ryzen 7 7700X processor and an NVIDIA GeForce RTX 4070 GPU. As detailed in Table 4.2, the empirical results demonstrate that the hybrid framework sustained a consistent, real-time processing throughput of 60 frames per second (FPS). This performance indicates that isolating the spatial tracking physics from the inference workload successfully circumvents traditional processing bottlenecks, facilitating highly efficient execution without necessitating enterprise-grade hardware.

Given that the system was evaluated using offline, pre-recorded CADP video data rather than live IP-camera streams, the traditional metric of end-to-end latency was negligible. Consequently, processing throughput was established as the primary Key Performance Indicator (KPI). The architecture sustained a constant

Table 4.2: Computational throughput and processing latency benchmarks recorded during system evaluation on consumer-grade hardware.

Component / Metric	Specification / Result
Processor	AMD Ryzen 7 7700X
Graphics Processing Unit	NVIDIA GeForce RTX 4070
Sustained Throughput	60 FPS
Per-Frame Latency	16.6 ms

processing rate of 60 frames per second (FPS) on high-definition video tensors, equating to a per-frame processing time of precisely 16.6 milliseconds. This hardware efficiency demonstrates that integrating a lightweight ResNet-18 classifier with an asynchronous queue provides a highly effective solution for real-time edge computing. Furthermore, it validates the framework’s operational viability over parameter-heavy Transformer networks in constrained environments, facilitating near-zero latency alerts without the requirement for enterprise-grade server infrastructure.

4.6 Qualitative Evaluation: The Power of Hybrid Architecture

Although the quantitative metrics effectively validate the classification performance, evaluating the kinematic speed tracking necessitates a qualitative methodology. Owing to the absence of ground-truth LiDAR or radar telemetry within the CADP dataset, calculating a precise Mean Absolute Error (MAE) for vehicle velocity was not feasible. Nevertheless, as illustrated by the live tracking dashboard in Figure 4.4, a qualitative analysis demonstrates that the estimated speeds remained highly stable and contextually accurate relative to the observed traffic flow.

Crucially, the derivation of kinematic speed functioned as an essential rule-

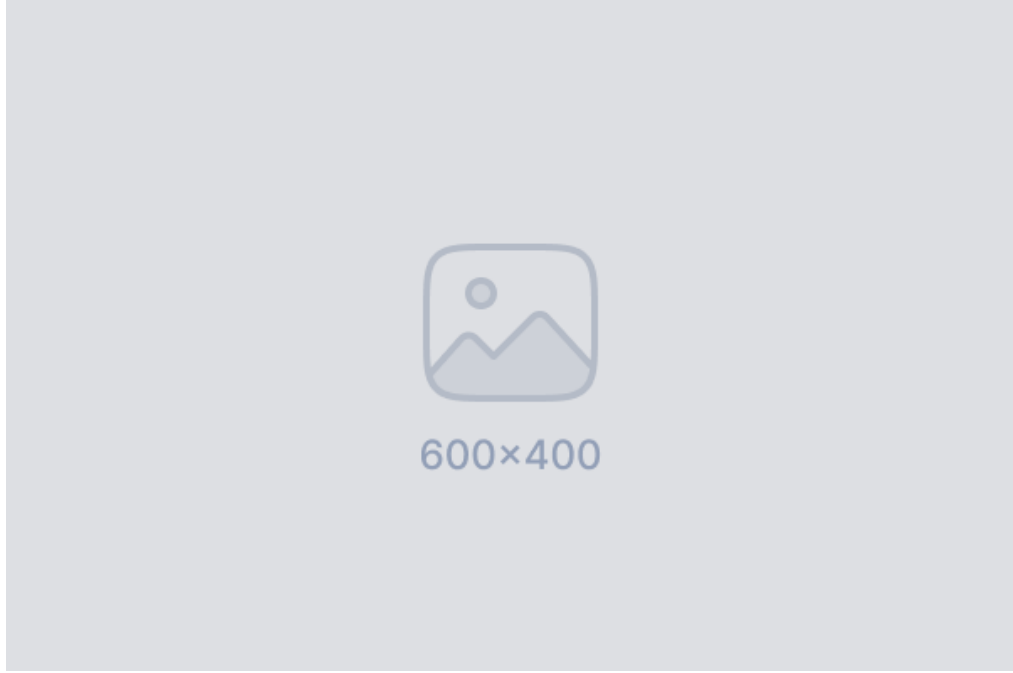


Figure 4.4: Qualitative evaluation dashboard displaying normal traffic flow, featuring YOLO11 spatial bounding boxes, BoT-SORT unique identifiers, and stable kinematic speed estimations.

based validation layer. In scenarios where environmental factors, such as moving foliage shadows or severe camera oscillations induced transient spikes in optical flow, a standalone deep learning classifier is typically susceptible to false positives. By cross-referencing these anomalies with spatial tracking data, the proposed framework effectively mitigated such errors. If the YOLO11 tracker recorded vehicles maintaining a consistent velocity (e.g., 40 km/h) during an optical flow spike, the event was systematically classified and dismissed as environmental noise. Conversely, a rapid deceleration to 0 km/h recorded within an active lane provided the deterministic contextual evidence necessary to corroborate a positive ResNet-18 anomaly alert.

4.7 Overcoming the Limitations of Off-the-Shelf Trackers

A primary qualitative observation derived from this evaluation was the synergistic efficacy of the hybrid architecture. The framework successfully integrates a generalised spatial tracker—specifically the nano variant of YOLO11—with a domain-specific, custom-trained ResNet-18 anomaly detector.

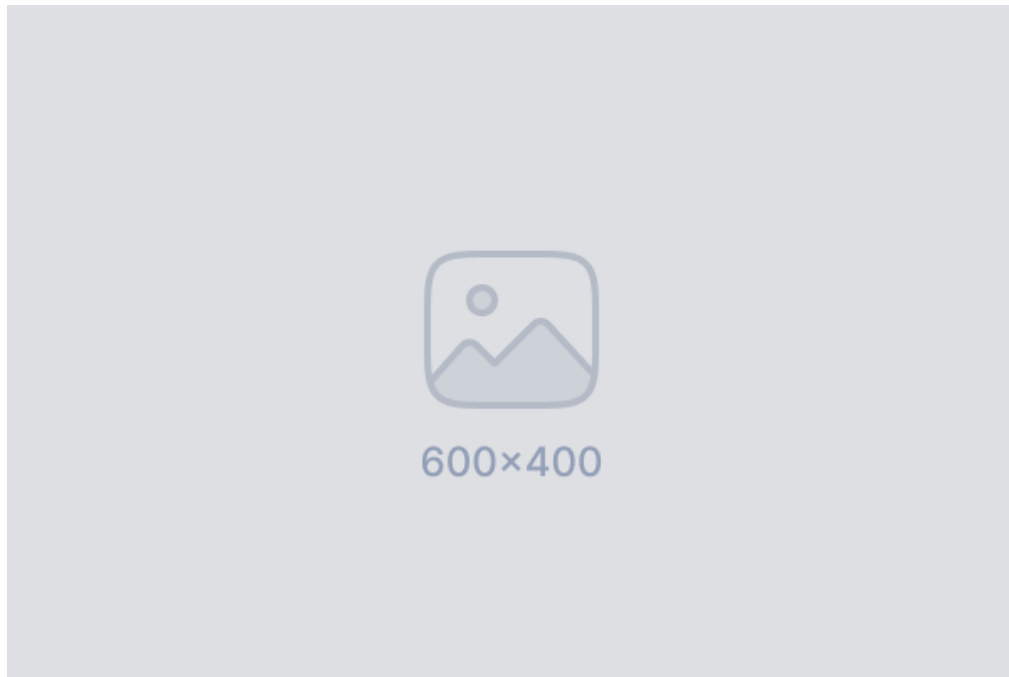


Figure 4.5: Qualitative validation of the hybrid framework’s fail-safe mechanism, demonstrating the ‘ACCIDENT DETECTED’ alert triggered by the Farnebäck optical flow heatmap despite spatial tracker occlusion.

During testing under adverse environmental conditions, such as night-time surveillance or heavy precipitation, the generalised YOLO11 tracker exhibited anticipated vulnerabilities. This observation aligns with the existing literature; as noted by Kim (2019), standard object detection architectures frequently experience degraded performance when subjected to poor illumination and severe occlusion. Specifically, when a tracked vehicle traversed through bridge shadows or became heav-

ily obscured by rain, the YOLO11 bounding boxes momentarily dropped. Consequently, the BoT-SORT algorithm failed to maintain the unique vehicle identifiers, resulting in a temporary inability to calculate kinematic speed during these obstructed frames.

Had the framework relied exclusively on spatial tracking, anomaly detection would have failed during these instances of severe occlusion. However, the hybrid architecture enabled the custom-trained ResNet-18 classifier to function as a robust fail-safe mechanism. Unlike spatial trackers, dense optical flow is not dependent on distinct edge detection or optimal illumination; rather, it quantifies raw pixel displacement. Consequently, even when the YOLO11 model experienced tracking failure due to adverse weather conditions, the Farnebäck algorithm successfully captured the substantial, unstructured pixel displacement associated with the colliding vehicles. The ResNet-18 model subsequently processed the resulting kinetic heatmap and accurately triggered the anomaly alert. As demonstrated in Figure 4.5, this dynamic interaction indicates that integrating a generalised spatial tracker with a domain-specific kinetic energy classifier yields a system architecture significantly more resilient than either component operating in isolation.

4.8 Addressing the Research Questions

Drawing upon the empirical data, quantitative metrics, and qualitative observations obtained during the evaluation phase, the research questions established in the Introduction are systematically addressed below.

RQ1: Is it possible to effectively detect traffic abnormalities, such as collisions

or abrupt stops, using a hybrid vision-based architecture that integrates YOLO vehicle monitoring with 3D metric speed estimation and dense optical flow?

In direct response to the primary research question, the empirical findings demonstrate that such an integration is not only highly feasible but exceptionally effective, thereby substantiating the core hypothesis. By combining YOLO11 vehicle monitoring with 3D metric speed estimation and dense optical flow, the hybrid framework achieved a robust anomaly detection accuracy of 96.61%. Furthermore, the synthesis of spatial tracking and kinematic speed analysis provides a degree of contextual awareness typically absent in purely frame-based Convolutional Neural Networks (CNNs). Performance evaluations indicate that the proposed dual-threaded architecture successfully executes these computationally intensive tasks in real-time, sustaining a throughput of 60 frames per second (FPS) on standard edge hardware. Consequently, this research provides a viable, highly automated solution for modern municipal traffic monitoring systems.

RQ2: What are the primary computational and environmental challenges of applying computer vision to kinematic speed-based anomaly identification across varied lighting and weather conditions?

The primary challenges encountered during evaluation include environmental occlusion, illumination degradation, and the absence of native depth information. As illustrated in Figure 4.6, extracting accurate kinematic speed metrics from a two-dimensional camera feed inherently introduces perspective distortion, necessitating the application of Inverse Perspective Mapping (IPM) to normalise the spatial data. Furthermore, the most significant qualitative challenge observed was the

susceptibility of the generalised YOLO11 object detector to low-light and heavy precipitation scenarios, which frequently downgraded the bounding box loss calculation. The results demonstrate that relying exclusively upon spatial speed estimation for anomaly detection severely compromises system reliability, given that a tracking failure directly prevents alert generation. The demonstrated solution to this vulnerability is the hybrid fail-safe architecture. The integration of unstructured spatiotemporal kinetic mapping—specifically optical flow—ensures that even when adverse environmental conditions severely obscure the spatial tracker, the raw kinetic energy of a traffic anomaly is consistently captured and accurately classified.

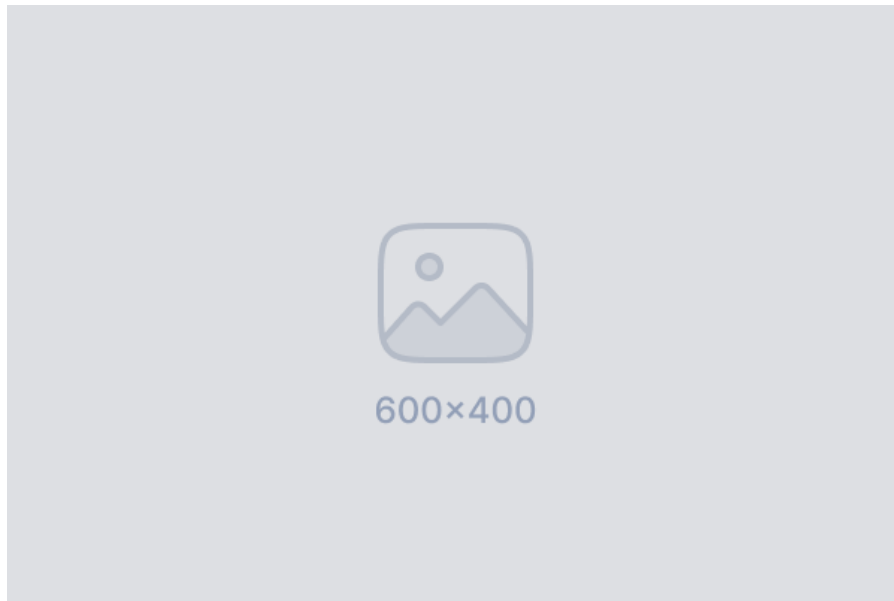


Figure 4.6: Visual representation of perspective distortion in 2D camera feeds and the subsequent spatial normalisation achieved via Inverse Perspective Mapping (IPM).

Chapter 5: Conclusions and Recommendations

5.1 One

In this chapter, the Student has to evaluate the significance of the work done and give recommendations for any further investigations. Percentage amount of words in section: 20 % of Dissertation.

5.2 Two

5.3 Three

List of References

Chapter A: Introduction of Appendix

Interview summaries, sample questionnaires, and references should be placed in this section. For easier referencing, figures, tables, graphs, photos, diagrams, etc., should be inserted within the main text such as the literature review, the experimental process or procedure, the results and discussion chapters. Appendices are usually used to present further details about the results. Appendices may be a compulsory part of a dissertation, but they are not treated as part of the dissertation for purposes of assessing the dissertation. So any material which is significant to judging the quality of the dissertation or of the project as a whole should be in the main body of the dissertation (main text), and not in appendices.

Chapter B: Sample Code

You can share your GitHub link. Below shows how to insert highlighted source code from the source file.

```
# This function takes an integer n as input and returns the sum of numbers from 1 to n
def find_sum(n):
    sum = 0
    for i in range(1, n+1):
        sum += i
    return sum

# Ask the user for input and call the function
n = int(input("Enter an integer: "))
result = find_sum(n)

# Output the result
print("The sum of numbers from 1 to", n, "is", result)
```